Manuel Vásquez 3/22/2026

WebDev Assigment

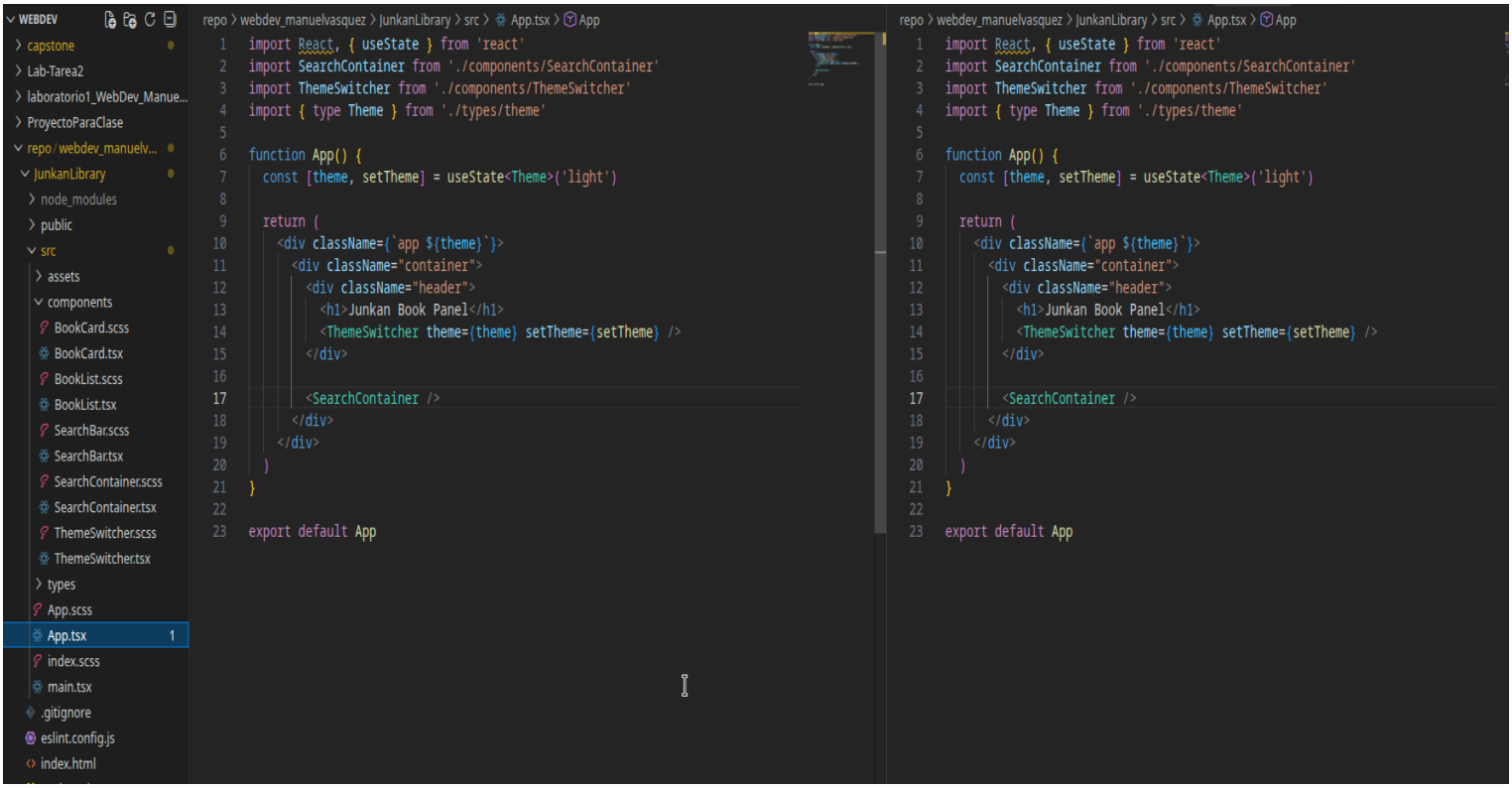
Comprensión Conceptos Fundamentales de React

Para esta tarea he utilizado de ejemplo mi implementación para el último laboratorio

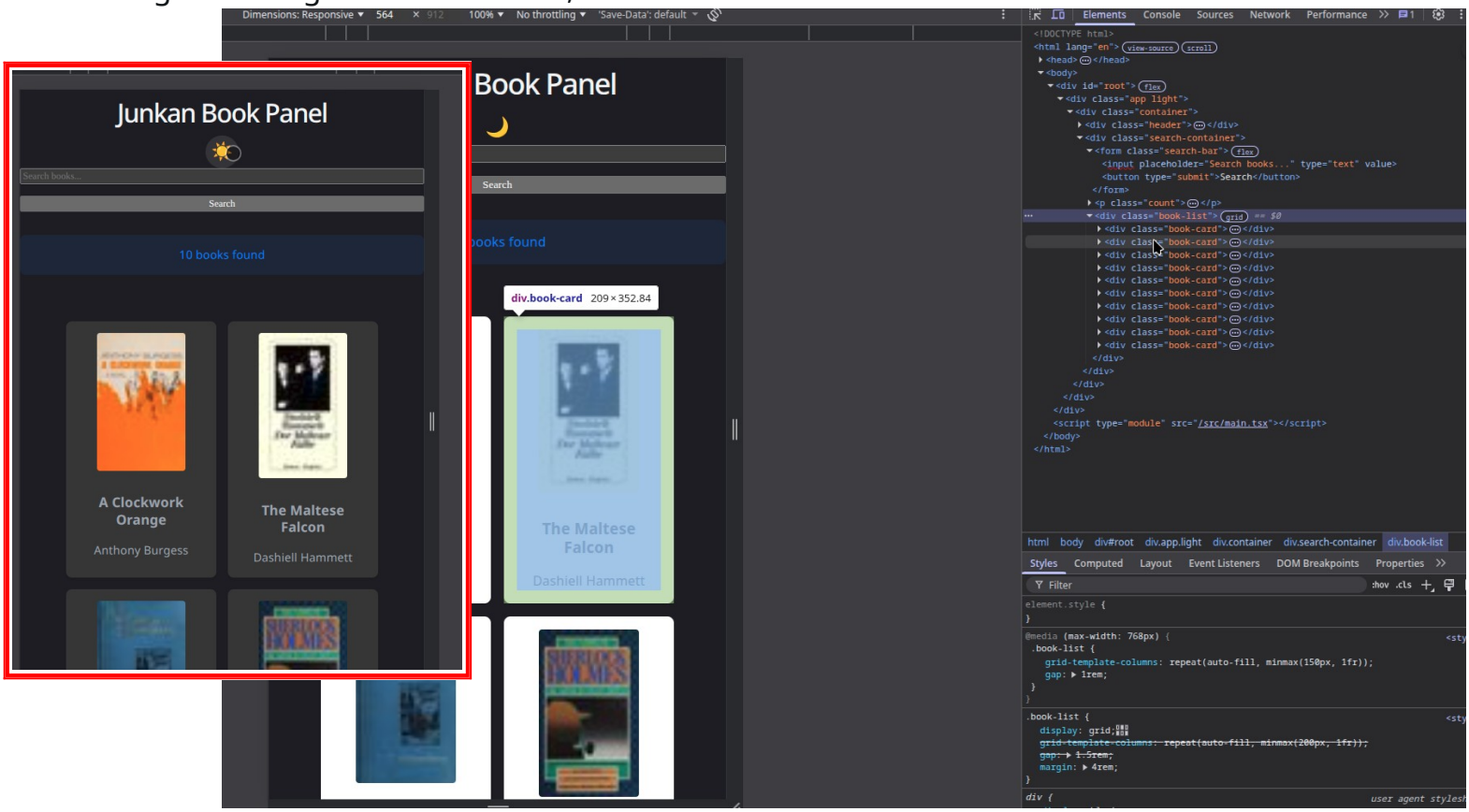
GitLab Link: https://gitlab.com/SerJunken/webdev_manuelvasquez/-/commits/LB-Tarea3-React-FundamentalConcepts

Es un ejemplo en donde utilizo la API de openLibrary para obtener libros y datos de ellos, buscar libros y tenerlos en una lista

Files and App.tsx



- Light And Night Mode on cards, final HTML Structure shown on dev view



•**Parte 1: Componentes y Props:** Analiza una relación de componente padre-hijo y explica el flujo de datos via props.

En esta aplicación de ejemplo, utilizo una relación de padre-hijo, parecida a una relación de smart y dumb component con mis componentes de SearchContainer y BookList (también con el searchComponent)

El usuario realiza una busqueda en el search bar

Entonces SearchContainer actualiza el estado que tiene de lista de books, y lo manda como prop al componente hijo BookList

SearchContainer.tsx .../webdev_manuelvasquez/... | SearchBar.tsx .../webdev_manuelvasquez/... | ThemeSwitcher.tsx .../webdev_manuelvasquez/... | ThemeSwitche

epo > webdev_manuelvasquez > JunkanLibrary > src > components > SearchContainer.tsx > SearchContainer

```
1  import { useState, useEffect } from 'react'
2  import SearchBar from './SearchBar'
3  import BookList from './BookList'
4  import type { Book, OpenLibraryResponse } from '../types/book'
5  import './SearchContainer.scss'
6
7  function SearchContainer() {
8    const [searchTerm, setSearchTerm] = useState<string>('')
9    const [books, setBooks] = useState<Book[]>([])
10   const [loading, setLoading] = useState<boolean>(false)
11
12   useEffect(() => {
13     if (!searchTerm) return
14
15     const fetchBooks = async () => {
16       setLoading(true)
17       try {
18         const res = await fetch(
19           `https://openlibrary.org/search.json?q=${searchTerm}&limit=10`
20         )
21         const data: OpenLibraryResponse = await res.json()
22         setBooks(data.docs || [])
23       } catch (error) {
24         console.error('Error:', error)
25         setBooks([])
26       } finally {
27         setLoading(false)
28       }
29     }
30
31     fetchBooks()
32   }, [searchTerm])
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

manuelvasquez@fedora:~/Documents/WebDev/repo/webdev_manuelvasquez$ git push origin --force

el componente BookList presenta los libros en una lista

```
  return (
    <div className="search-container">
      <SearchBar onSearch={setSearchTerm} />

      {loading && <p className="loading">Loading...</p>}

      {!loading && books.length > 0 && (
        <>
          <p className="count"> {books.length} books found</p>
          <BookList books={books} />
        </>
      )}

      {!loading && searchTerm && books.length === 0 && (
        <p className="no-results">No books found</p>
      )}
    </div>
  )
}
```

```
import BookCard from './BookCard'
import { type Book } from '../types/book'

import './BookList.scss'

interface BookListProps {
  books: Book[]
}

function BookList({ books }: BookListProps) {
  return (
    <div className="book-list">
      {books.map((book, index) => (
        <BookCard key={book.key || index} book={book} />
      ))}
    </div>
  )
}

export default BookList
```

Los props son unidireccionales, solo van del padre al hijo

El componente hijo no puede modificar las props que recibe

Los componentes hijos solo presentan los objetos que recibe, no contienen mucha lógica compleja.

(BookListProps)

```
import BookCard from './BookCard'
import { type Book } from '../types/book'

import './BookList.scss'

interface BookListProps {
  books: Book[]
}

function BookList({ books }: BookListProps) {
  return (
    <div className="book-list">
      {books.map((book, index) => (
        <BookCard key={book.key || index} book={book} />
      ))}
    </div>
  )
}

export default BookList
```

•**Parte 2: Estado y Eventos:** Analiza un componente con estado y explica cómo useState y los eventos trabajan juntos para crear interactividad.

```
const [input, setInput] = useState<string>('')
```

```
function SearchBar({ onSearch }: SearchBarProps) {
  const [input, setInput] = useState<string>('')
```

El componente SearchBar utiliza el useState y los eventos crean la interactividad

```
const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  setInput(e.target.value)
}
```

Cada vez que el usuario escribe,el evento onChange captura el valor del input y actualiza el estado

```
const handleSubmit = (e: React.FormEvent<HTMLFormElement>) => {
  e.preventDefault()
  if (input.trim()) {
    onSearch(input.trim())
    setInput('')
  }
}
```

Cuando el usuario envía el formulario, el evento handleSubmit, maneja que la pagina no se recarge con preventDefault, verifica que el input no sea vacío, y llama a la función onSearch, pasando el valor al estado, (ya que el estado puede ser leido por el componente padre)

```
return (
  <div className="search-container">
    <SearchBar onSearch={setSearchTerm} />
```

```
import './SearchContainer.scss'

function SearchContainer() {
  const [searchTerm, setSearchTerm] = useState<string>('')
  const [books, setBooks] = useState<Book[]>([])
  const [loading, setLoading] = useState<boolean>(false)

  useEffect(() => {
```

- **Parte 3: Efectos Secundarios y Renderizado:** responde preguntas conceptuales sobre elegir el hook correcto e implementar renderizado condicional.

¿Por qué usar useEffect para llamadas a API?

Utilicé useEffect como hook para las llamadas API porque conceptualmente son efectos secundarios, ocurren fuera del renderizado de React, por eso useEffect maneja esas operaciones

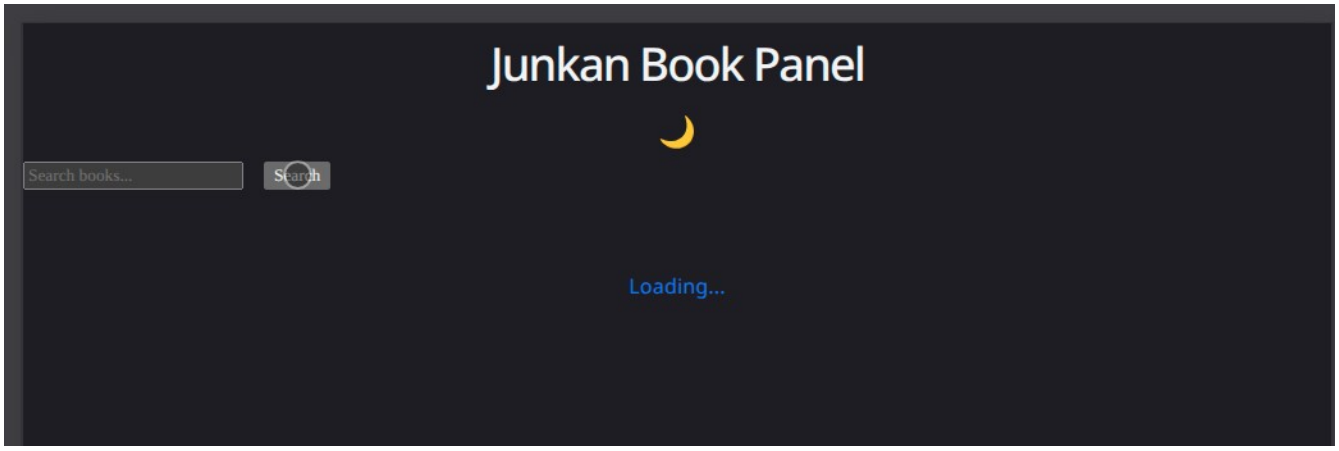
```
function SearchContainer() {
  const [searchTerm, setSearchTerm] = useState<string>('')
  const [books, setBooks] = useState<Book[]>([])
  const [loading, setLoading] = useState<boolean>(false)

  useEffect(() => {
    if (!searchTerm) return

    const fetchBooks = async () => {
      setLoading(true)
      try {
        const res = await fetch(
          `https://openlibrary.org/search.json?q=${searchTerm}&limit=10`
        )
        const data: OpenLibraryResponse = await res.json()
        setBooks(data.docs || [])
      } catch (error) {
        console.error('Error:', error)
        setBooks([])
      } finally {
        setLoading(false)
      }
    }

    fetchBooks()
  }, [searchTerm])
```

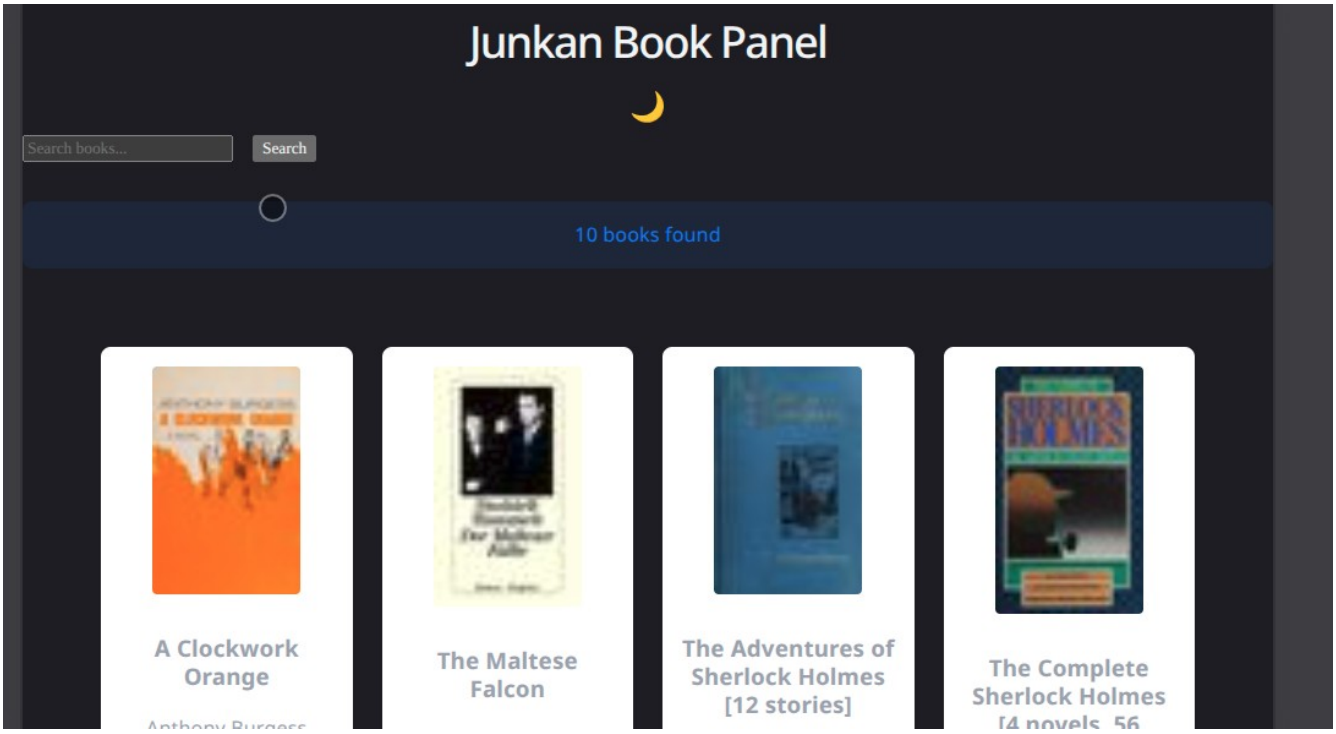
Agrego en el array de dependenciasl el searchTerm, además, manejo los errores con un try catch, y manejo el estado de loading hacia false cuando termine.

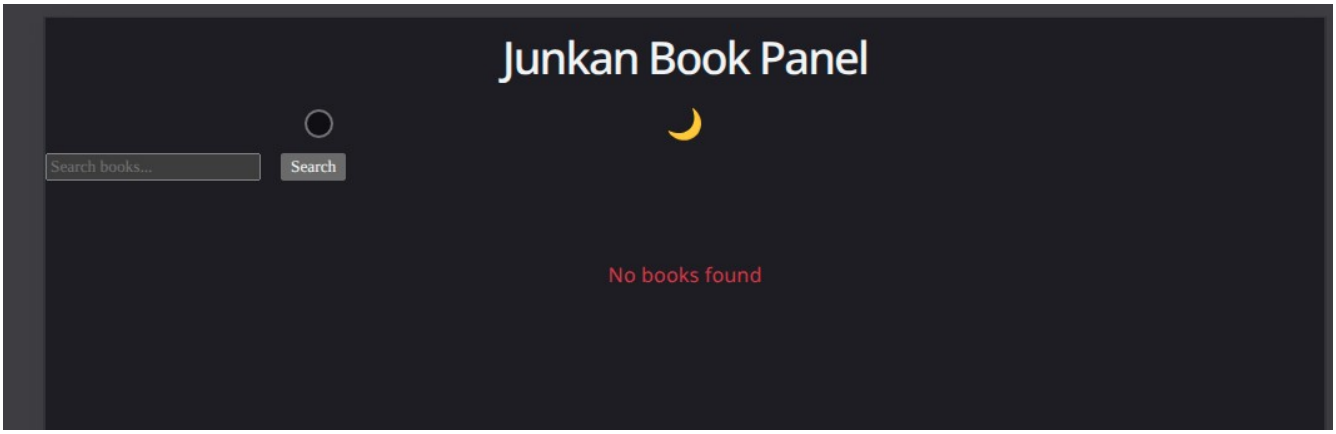
Manuel Vásquez 3/22/2026

¿Cuándo usar renderizado condicional?
Implemento renderizado condicional para mostrar diferentes estados de la interfaz; Mientras la API está cargando, muestro “Loading…” condicionalmente, si hay libros, los renderizo, si no, muestro el mensaje, si no hay busqueda, no muestro nada

```
return (
  <div className="search-container">
    <SearchBar onSearch={setSearchTerm} />

    {loading && <p className="loading">Loading...</p>}

    {!loading && books.length > 0 && (
      <>
        <p className="count"> {books.length} books found</p>
        <BookList books={books} />
      </>
    )}

    {!loading && searchTerm && books.length === 0 && (
      <p className="no-results">No books found</p>
    )}
  </div>
)
```







También enel bookcard, si el libro tiene cover, entonces se muestra la img con el class book-cover, si no, se muestra un div de clase no-cover que manejo en los estilos

```
return (
  <div className="book-card">
    {coverUrl ?
      <img className="book-cover" src={coverUrl} alt={title} />
    ) : (
      <div className="no-cover"></div>
    )}
    <h3>{title}</h3>
    <p>{author}</p>
  </div>
```

```
.no-cover {
  width: 100%;
  height: 150px;
  background-color: #f0f0f0;
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 3rem;
  border-radius: 4px;
  margin-bottom: 0.5rem;
}

.book-cover {
  width: 70%;
  height: auto
}
```